

problem about Paramsolver in cirq and in QTF

<https://github.com/tensorflow/quantum/issues/646>

ROW 70

problem about Paramsolver in cirq and in QTF #646

New issue

Closed



LeeQY1996 opened on Dec 11, 2021

...

I have a problem about Paramsolver in cirq and in QTF. I construct a hardware efficient circuit to do VQE algorithm. I use QTF to generate a `tf.keras.models.Model` and optimize this circuit to get optimal parameter values and estimation energy value. I use three methods to test this parameter values. Firstly, I use `set_weights` to set optimal parameters to this QTF.model and get the estimation value. The two way get the same value. And I use other two methods, `tfq.layers.Expectation()` and `simulator.simulate_expectation_values()` to do the same calculation. But I find that the same optimal parameters and the same circuit, I get different estimation values. The QTF.model and `set_weights` are the same values. And `tfq.layers.Expectation()` and `simulator.simulate_expectation_values()` are the same values. But if the circuit only contains 8 parameters or less, The four methods get the same values.

```
parameters_number:8, QTF_value:-0.4172770082950592, Reset_Weights_QTF_Value:[[-0.417277]],
QTF.Expectation_value:[[-0.417277]], Simulator_value:[(-0.4172773063182831+0j)]
```

```
parameters_number:16, QTF_value:-4.257663726806641, Reset_Weights_QTF_Value:[[-4.2576637]],
QTF.Expectation_value:[[-1.1815777]], Simulator_value:[(-1.1815786734223366+7.450580596923828e-09j)]
```

My code is available [here](#)



Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Participants



lockwo on Dec 11, 2021 · edited by lockwo

Edits Contributor

That's because the internal ordering of the PQC is different. I.e. if we compare the PQC layer's `._symbols` with the `params_str` we can see the difference. `._symbols = tf.Tensor([b'vqe0' b'vqe1' b'vqe10' b'vqe11' b'vqe12' b'vqe13' b'vqe14' b'vqe15' b'vqe2' b'vqe3' b'vqe4' b'vqe5' b'vqe6' b'vqe7' b'vqe8' b'vqe9'], shape=(16,), dtype=string)` where as `params_str = tf.Tensor([b'vqe0' b'vqe1' b'vqe2' b'vqe3' b'vqe4' b'vqe5' b'vqe6' b'vqe7' b'vqe8' b'vqe9' b'vqe10' b'vqe11' b'vqe12' b'vqe13' b'vqe14' b'vqe15'], shape=(16,), dtype=string)`. Hence when you are getting the weights (which are in the order of the PQC's, so it would be `weight_0 weight_1 weight_10`) and trying to plug them into a different ordering, you get different results.

If you want to keep this approach (and use expectation layers with pretrained weights) you can use the logic that the PQC uses to get the ordering:

```
self._symbols_list = list(
    sorted(util.get_circuit_symbols(model_circuit)))
self._symbols = tf.constant([str(x) for x in self._symbols_list])
```



I.e. you can get the exact outputs by simply doing the following at the end of the code:

```
# These are the only two lines that are different
params_str = list(sorted(tfq.util.get_circuit_symbols(circuit)))
params = [sympy.symbols(i) for i in params_str]

qev=tfq.layers.Expectation()(circuit,
                             symbol_names=params_str,
                             symbol_values=[theta],
                             operators=observable)

#using simulator to get the estimation energy value.
simulator=Simulator()
resolver=Param(theta,params) #参数器
sv=simulator.simulate_expectation_values(circuit,observable,resolver)
```



And this will yield and output of: `parameters_number:16, QTF_value:-4.212678909301758, Reset_Weights_QTF_Value:[[-4.212679]], QTF.Expectation_value:[[-4.212679]], Simulator_value:[(-4.212678354233503+0j)]`



1

```
sorted(util.get_circuit_symbols(model_circuit)))
self._symbols = tf.constant([str(x) for x in self._symbols_list])
```

I.e. you can get the exact outputs by simply doing the following at the end of the code:

```
# These are the only two lines that are different
params_str = list(sorted(tfq.util.get_circuit_symbols(circuit)))
params = [sympy.symbols(i) for i in params_str]

qev=tfq.layers.Expectation()(circuit,
                             symbol_names=params_str,
                             symbol_values=[theta],
                             operators=observable)

#using simulator to get the estimation energy value.
simulator=Simulator()
resolver=Param(theta,params) #参数器
sv=simulator.simulate_expectation_values(circuit,observable,resolver)
```

And this will yield and output of: parameters_number:16, QTF_value:-4.212678909301758, Reset_Weights_QTF_Value:[[-4.212679]], QTF.Expectation_value:[[-4.212679]], Simulator_value:[{-4.212678354233503+0j}]



LeeQY1996 on Dec 12, 2021

Author ...

Thank you very much for helping me solve this very important problem.



🔒 LeeQY1996 closed this as completed on Dec 19, 2021

📄 lockwo mentioned this on Jan 14, 2023

🔗 [Can we use two PQC or ControlledPQC layers in same Sequential class? #737](#)

Can we use two PQC or ControlledPQC layers in same Sequential class? #737

New issue

Open



Shuhul24 opened on Jan 14, 2023 · edited by Shuhul24

Edits · ...

I want two different quantum circuits, H1 and H2, in same `Sequential` class, similar to two `tf.keras.layers.Dense` that have different weight values, bias values and number of neurons as well. Is it possible using `tfq.layers` ?



lockwo on Jan 14, 2023

Contributor · ...

Why make them as different TFQ layers? If you do circuit H1 then circuit H2 you can just combine them in `cirq` then use one PQC



Shuhul24 on Jan 14, 2023

Author · ...

Hi. Thanks for the reply!

I want to evaluate the gradients with respect to each layer weights (parameters of quantum gates), which is possible if I declare two different PQC layers.

In first PQC layer, say H1, I want the output of that layer to be the input for the next PQC layer, H2 and not its observable as the input to next PQC layer. The output of a layer can be evaluated by `tfq.layers.State` but I think its not possible to use that output state as an input for next PQC layer. Also mentioned that the output state through `tfq.layers.State` is not differentiable (I am not sure about this), which kind of creates another issue for evaluating gradient.



lockwo on Jan 14, 2023

Contributor · ...

I'm not sure I understand your issue. First, it is worth noting that you can get the gradients with respect to different parts of the circuit within a single PQC. Just do

```
with tf.GradientTape() as tape:
    energy = model(inputs)
    grads = tape.gradient(energy, model.trainable_variables)
    h1_grads = grads[0:len(h1_params)]
    h2_grads = grads[len(h1_params):]
```



Or something similar (that is more pseudo code than real code).

The reason I am confused is because I think what you are saying is you want to do H1 -> `tfq.layers.State()` -> H2. You are correct in saying that state is not differentiable. My confusion comes from the fact that that is what circuits just do by default, if you have H1 (`cirq.Circuit`) and H2 (`cirq.Circuit`) and do H1 + H2 then use that as a PQC that is the exact same as H1 -> State -> H2 (since the quantum state). You could also get the gradients for just part of this by freezing certain parameters or you just also just get all the gradients and only use those for H1 or H2 for what you need.

Now you could also be asking how I do H1 -> observable -> `controlledPQC(H2)`, which should be the same as in the demo in hello many worlds. You just specify the `controlledPQC` param weights to be the output and it should work fine.



Shuhul24 on Jan 14, 2023

Author · ...

Thanks for the reply!

Is it possible to get the weights of different sections within single PQC, as the weights are `tf.Variable` format and I can't get them just by slicing using `tf.slice`.

Assignees

No one assigned

Labels

kind/question

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Participants

